

Arrays Prog00_Imp_Progs Explanation

Array Notes (Striver A2Z DSA)

1. Left Rotate Array by One Place

Problem

Rotate an array to the left by one position.

Example

```
Input:
1 2 3 4 5

Output:
2 3 4 5 1
```

Intuition

- Store first element.
 - Shift every element one position to the left.
 - Place stored element at the end.
-

Algorithm

```
temp = arr[0]

for i = 1 to n-1
    arr[i-1] = arr[i]
```

```
arr[n-1] = temp
```

Complexity

Time

```
O(n)
```

Extra Space

```
O(1)
```

2. Left Rotate Array by D Places

Important Observation

If

```
D >= N
```

then

```
D = D % N
```

Reason:

Rotating N times gives the original array.

Example

```
N = 7
```

```
Rotate 15 times
```

```
15 % 7 = 1
```

```
Only 1 rotation needed.
```

Brute Force

Steps

1. Store first D elements in temporary array.
2. Shift remaining elements left.
3. Copy temporary elements to end.

Algorithm

```
D = D % N
```

```
Store first D elements
```

```
Shift remaining elements left
```

```
Copy temp to end
```

Complexity

Time

```
O(D)  
+  
O(N-D)  
+  
O(D)  
  
= O(N+D)
```

Extra Space

$O(D)$

Optimal (Reversal Algorithm)

Idea

Example

1 2 3 4 5 6 7

$D = 3$

Step 1

Reverse first D

3 2 1 4 5 6 7

Step 2

Reverse remaining

3 2 1 7 6 5 4

Step 3

Reverse entire array

4 5 6 7 1 2 3

Algorithm

```
Reverse(0, D-1)
```

```
Reverse(D, N-1)
```

```
Reverse(0, N-1)
```

Complexity

Time

```
O(N)
```

Extra Space

```
O(1)
```

Right Rotate by D Places

Interview Assignment

Hint:

```
Reverse last D
```

```
Reverse first N-D
```

```
Reverse whole array
```

3. Move Zeros to End

Example

```
1 0 2 3 0 4 0 5
```

↓

1 2 3 4 5 0 0 0

Maintain order of non-zero elements.

Brute Force

Steps

- Store all non-zero elements.
 - Copy them to front.
 - Fill remaining positions with zero.
-

Complexity

Time

$O(N)$

Extra Space

$O(N)$

Worst case: no zeros.

Optimal (Two Pointer)

Idea

Find first zero.

Maintain

```
j = first zero
```

Traverse remaining array

Whenever

```
arr[i] != 0
```

Swap

```
arr[i]  
and  
arr[j]
```

Increment

```
j++
```

Algorithm

```
Find first zero index j  
for i = j+1 to N-1  
    if arr[i] != 0  
        swap(arr[i], arr[j])  
        j++
```

Complexity

Time

$O(N)$

Extra Space

$O(1)$

4. Linear Search

Problem

Find first occurrence of target.

Example

Array

2 4 7 8 9

Target = 8

Answer = 3

Algorithm

```
for i = 0 to N-1
    if arr[i] == target
        return i
return -1
```

Complexity

Time

$O(N)$

Space

$O(1)$

5. Union of Two Sorted Arrays

Example

A

1 2 3 4 5

B

2 3 4 5 6

Union

1 2 3 4 5 6

No duplicates.

Brute Force (Using Set)

Steps

- Insert all elements of both arrays into a Set.
- Copy Set into answer array.

Complexity

Time

$O(N_1 \log N)$

+

$O(N_2 \log N)$

Space

$O(N_1 + N_2)$

Optimal (Two Pointer)

Since arrays are sorted.

Maintain

i
j

Compare

- Smaller element goes into answer.
- If equal, insert once.
- Ignore duplicates.

Continue until one array ends.

Then process remaining array.

Complexity

Time

$O(N_1 + N_2)$

Space

$O(1)$

(Excluding output array.)

6. Intersection of Two Sorted Arrays

Example

```
A
1 2 2 3 3 5 6

B
2 3 3 5 6

Intersection
2 3 3 5 6
```

Duplicates are allowed if present in both.

Brute Force

Idea

For every element in first array

Search matching unused element in second array.

Maintain

```
visited[]
```

Complexity

Time

```
O(N1 × N2)
```

Space

```
O(N2)
```

Optimal (Two Pointer)

Maintain

```
i
```

```
j
```

Cases

Case 1

```
A[i] < B[j]
```

```
Move i
```

Case 2

```
A[i] > B[j]
```

```
Move j
```

Case 3

```
Equal
```

Store answer

Move both

Complexity

Time

$O(N1 + N2)$

Space

$O(1)$

Important Interview Tips

Always Explain Approaches in Order

Brute Force

↓

Better (if exists)

↓

Optimal

Interviewers care about your thought process.

Clarify Space Complexity

Say clearly:

- **Algorithm space** includes the given input array if discussing total memory used.

- **Extra space** only counts additional memory you allocate.

Example

```
Left Rotate by One
```

```
Extra Space = 0(1)
```

For Rotation Problems

Always reduce rotations:

```
D = D % N
```

Sorted Arrays

Whenever arrays are sorted, think:

```
Two Pointer
```

before considering nested loops.

Key Takeaways

- Left Rotate by 1 → Store first element and shift.
- Left Rotate by D → Use `D % N`.
- Optimal rotation → Three reversals.
- Move Zeros → Two-pointer swapping.
- Linear Search → Single traversal.
- Union of Sorted Arrays → Two pointers (optimal).
- Intersection of Sorted Arrays → Two pointers (optimal).
- Always explain Brute → Better → Optimal in interviews.

- Be precise about **extra space** vs **total space**.